Analyse de l’application Refuges

Akrem SOUSSI et Rémi CORTIAL · Polytech Marseille · DEV 1

1. Objectif et architecture

Refuges est une application communautaire de randonnée. Elle permet de rechercher un refuge sur une carte, de consulter ses informations, de partager des publications dans des communautés et de préparer des sorties. La base contient 4 652 refuges issus de refuges.info. Les informations et les fonds OpenStreetMap sont accompagnés de leurs attributions.

L’application suit une architecture trois tiers. Le navigateur affiche le client React/TypeScript, servi par Nginx. Les requêtes /api/ sont transmises à FastAPI, qui applique les règles métier et accède à MySQL via SQLAlchemy. Le navigateur ne dialogue jamais directement avec la base. Les déploiements VM et Docker conservent cette séparation.

2. Acteurs et droits

Acteur	Fonctions accessibles	Limites principales
Visiteur	Connexion, inscription, confirmation d’email, récupération du mot de passe, conditions et statut du service.	Les pages applicatives nécessitent une session active. Un compte en attente reste limité à la confirmation.
Utilisateur (user)	Carte, refuges, communautés, publications, sorties, amis, profil, confidentialité et signalements.	Aucun accès à la modération. Modification de ses propres contenus uniquement.
Administrateur (moderator)	Fonctions utilisateur, traitement des signalements, gestion des communautés et des comptes ordinaires.	Ne peut pas changer les rôles ni agir sur les comptes administrateurs.
Administrateur supérieur (super_admin)	Fonctions précédentes, promotion/rétrogradation des modérateurs et gestion de leurs comptes.	Ne peut pas suspendre, supprimer ou rétrograder son propre compte.

Les restrictions sont appliquées par les gardes de routes du client et par les contrôles d’autorisation du serveur. Masquer un bouton ne remplace pas une vérification côté API : une requête interdite doit également être rejetée.

3. Principes de conception

La navigation principale regroupe Carte, Communautés et Sorties ; la modération apparaît pour les rôles autorisés. Les styles partagés utilisent Inter, un vert principal #21664a et des fonds clairs. Les formulaires et listes prévoient des états de chargement, d’erreur et de résultat vide.

La maquette sert de référence pour la structure et les composants. Le site ajoute les données réelles, les sessions, les contrôles métier et les interactions cartographiques. Notre utilisation de l’IA est décrite séparément dans la démarche Figma.

4. Cas d’utilisation : accès et consultation

CU1 - S’inscrire et se connecter

Acteur et déclencheur : un visiteur soumet le formulaire d’inscription ou de connexion.

Scénario nominal : le visiteur renseigne un pseudo unique, son email et un mot de passe d’au moins huit caractères, puis accepte les conditions. Le serveur conserve un condensat Argon2id du mot de passe, crée un compte en attente et un jeton de confirmation. La validation du lien active le compte. Une connexion valide crée une session serveur et un cookie refuges_session HttpOnly, SameSite=Lax.

Erreurs : pseudo ou email déjà utilisé : HTTP 409 ; jeton expiré ou déjà utilisé : HTTP 400 ; identifiants erronés : HTTP 401 avec le même message générique ; compte suspendu : HTTP 403. Un compte non confirmé reste sur l’écran d’attente.

CU2 - Réinitialiser son mot de passe

Acteur et déclencheur : un visiteur sélectionne « Mot de passe oublié » et renseigne son email.

Scénario nominal : pour un compte admissible, le serveur génère un lien temporaire valable une heure. La validation du nouveau mot de passe remplace son condensat et révoque toutes les anciennes sessions.

Erreurs : un lien invalide ou expiré est rejeté en HTTP 400. La demande initiale reçoit une réponse identique, que l’adresse soit connue ou non, afin de ne pas révéler les comptes inscrits.

CU3 - Rechercher et consulter un refuge

Acteur et déclencheur : un utilisateur connecté ouvre la carte, déplace son emprise ou saisit une recherche.

Scénario nominal : le client demande les refuges correspondant à l’emprise et aux filtres. Les marqueurs sont regroupés et une liste accompagne la carte. La sélection d’un refuge donne accès à sa fiche : altitude, capacité, gardiennage, description, accès, photos disponibles, publications et sorties associées.

Erreurs : une recherche sans résultat affiche un état vide ; un identifiant inexistant renvoie HTTP 404. L’absence de photographie est représentée par un emplacement explicite.

Exemple de parcours réalisé : carte → recherche « Pelvoux » → sélection → fiche du refuge. Les captures du parcours montrent le résultat dans l’application locale.

5. Cas d’utilisation : communauté et administration

CU4 - Publier dans une communauté

Acteur et déclencheur : un utilisateur membre ouvre « Nouveau post ».

Scénario nominal : il choisit la communauté, saisit son texte et peut associer un refuge, jusqu’à quatre photos et une trace GPX. Il choisit une visibilité publique ou limitée aux amis. Les images sont réencodées en WebP et nettoyées de leurs métadonnées EXIF. Le serveur calcule la distance de la trace et son dénivelé positif lorsque des altitudes sont disponibles.

Erreurs : une publication vide ou une image invalide est rejetée en HTTP 422 ; publier dans une communauté sans en être membre est interdit en HTTP 403.

Parcours illustré : communautés → Randonneurs des Écrins → nouveau post → publication textuelle → vérification après rechargement. Cette démonstration ne couvre pas l’import de photo ou de GPX.

CU5 - Organiser et rejoindre une sortie

Acteur et déclencheur : un utilisateur crée une sortie ou ouvre une sortie pour y participer.

Scénario nominal : l’organisateur définit le titre, le départ, une capacité de 2 à 50 participants, la visibilité et au moins une étape. Une trace peut être ajoutée. L’organisateur devient le premier participant ; les autres utilisateurs admissibles peuvent rejoindre ou quitter la sortie.

Erreurs : une sortie complète ou passée refuse une nouvelle participation en HTTP 409. Une sortie sur invitation est masquée ou refusée aux utilisateurs non invités. Les modifications sont contrôlées selon l’identité et les droits de l’utilisateur.

CU6 - Modérer les contenus et les comptes

Acteur et déclencheur : un modérateur ou un administrateur supérieur ouvre l’espace de modération.

Scénario nominal : il examine les signalements, traite les contenus concernés et gère les communautés. La suspension d’un compte révoque ses sessions et empêche sa reconnexion. L’administrateur supérieur peut aussi attribuer ou retirer le rôle de modérateur. Les actions sont consignées dans le journal de modération.

Erreurs : une action dépassant les droits du modérateur est rejetée en HTTP 403. Une action interdite sur son propre compte est rejetée en HTTP 409.

6. Vérification fonctionnelle

Le cahier de recette décrit les cas nominaux, les erreurs et les contrôles des quatre profils. Les deux parcours photographiés le 26 octobre complètent cette recette historique ; ils ne constituent pas une nouvelle exécution de tous les tests. Les essais de charge JMeter font l’objet d’un rapport distinct.

7. Règles de gestion et modèle relationnel

Règle	Traduction dans les données et le métier
Identité et statut	users impose l'unicité du pseudo et de l'email. Rôle et statut sont limités aux valeurs prévues.
Authentification	Mots de passe sous forme de condensats Argon2id ; jetons de session et d'email sous forme d'empreintes SHA-256. L'auteur d'une action est déterminé par la session serveur.
Refuges et massifs	refuges.external_id est unique. Un refuge peut être lié à un massif et à plusieurs photos ordonnées.
Communautés	Le slug est unique. community_refuges représente l'association multiple entre communautés et refuges.
Adhésions et réactions	Les clés composées de community_members et post_likes empêchent les doublons d'adhésion ou de réaction.
Amitiés	Une contrainte empêche les doublons dans un même sens ; le métier vérifie aussi la relation inverse. Les audiences règlent l'accès aux informations privées.
Sorties	Un organisateur, des étapes ordonnées et des participants uniques ; capacité, date et invitations sont contrôlées par le serveur.
Suppression et journal	Les clés étrangères assurent les suppressions dépendantes prévues. Le journal de modération est conservé, avec un acteur nullable si son compte disparaît.

8. Correspondance entre maquette et application

Écrans Figma	Pages et routes principales
01, 02, 02b - Accès	/connexion, /inscription, /inscription/confirmation ; confirmation d'email et récupération du mot de passe.
03, 04 - Refuges	/carte, /refuges/:id.
05, 05b, 05c, 06, 06b - Profils	/profil, /parametres/amis, /parametres/confidentialite, /u/:username ; affichage selon les droits de visibilité.
07, 08, 09 - Communautés	/communautes, /communautes/:slug, /posts/nouveau.
10, 11, 12 - Sorties	/sorties, /sorties/:id, /sorties/nouvelle, modification d'une sortie.
13 à 16 - Modération	/moderation, signalements, communautés et comptes ; accès selon le rôle.

Maquette Figma. Les identifiants détaillés des écrans, les règles SQL et les scénarios étendus restent disponibles dans l'annexe d'analyse. Les diagrammes décrivent la base, les cas d'utilisation, les classes, les séquences et l'architecture. Les exemples API et la recette complètent le dossier.